Lab 4: Buffer Overflow Principles

Exercise Quick-Links

Goal: In this lab, you will study the basic principles that lead to buffer overflow attacks.

Getting Started

We've offered some code here to start with. Download this code to your machine and unpack it into some directory (unrar x lab1-code.rar).

Now, you should browse the source code we given and find out the file stack1.c. There is a simple C program in this file, which has buffer overflow vulnerability. You can compile this program using the gcc compiler:

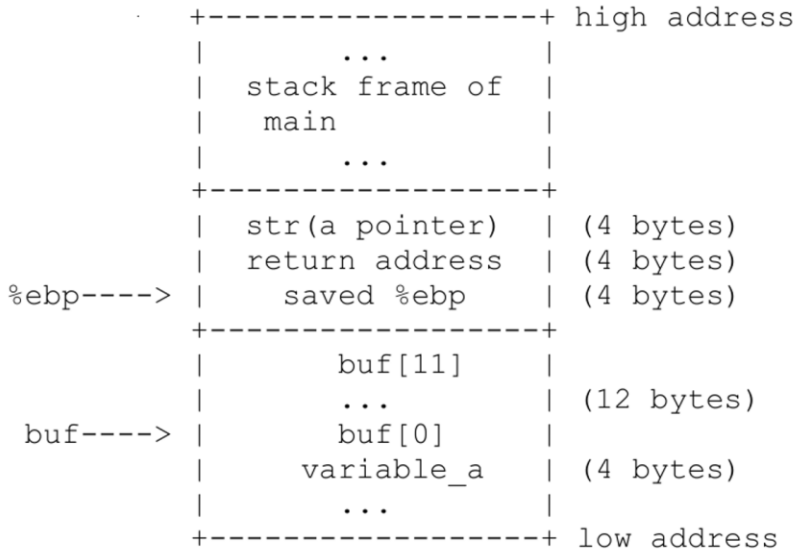
```
$ gcc -g stack1.c -o stack1
$ ./stack1
```

note the use of the -g parameter, which will be useful when you debug the executable using gdb.

Stack Layout and Buffers

In computer science, a call stack is a stack-like data structure holding information to control function calls and returns. Stack layout is the convention on how the stack frame is used. As an example, read the simple C program given you (the C file stack1.c).

When the function main calls the function func, the stack layout looks like the following, pay special attention to the positions of the local variables and arguments.



Exercise 1.

Now, you can write some code. Your job is to print the address of the variable buffer, in the C program stack1.c, and compile the C program as above. Run it three times, observe and write down the output addresses in address.txt, are these 3 addresses the same or not?

Add line to stack1.c: printf("Address of buffer: %p\n", (void*)buffer);

```
┌──(kali㉿kali)-[~/buffer-overflow-principles/lab1-code]
└─$ ./stack1
Address of buffer: 0×7ffd10a54b64
Returned Properly

┌──(kali㉿kali)-[~/buffer-overflow-principles/lab1-code]
└─$ ./stack1
Address of buffer: 0×7ffcf65fc844
Returned Properly

┌──(kali㉿kali)-[~/buffer-overflow-principles/lab1-code]
└─$ ./stack1
Address of buffer: 0×7ffd17014b34
Returned Properly
```

The address of the variable was different each time stack1.c was compiled since Linux implement ASLR & randomizes the memory layout of each program every time it runs (including stack/heap addresses and shared library locations).

Now you can investigate the stack layout and C calling convention in detail, for this, you should use the debugger gdb.

Exercise 2.

Use gdb to debug the program, as the following. You may find the online gdb manual http://www.sourceware.org/gdb/current/onlinedocs/gdb/ useful.

$ gdb stack1
(gdb) b func

```
Breakpoint 1 at 0×1165: file stack1.c, line 10.
```

(gdb) r

```
Starting program: /home/kali/buffer-overflow-principles/lab1-code/stack1
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, func (str=0×55555555601b "hello\n") at stack1.c:10
10          printf("Address of buffer: %p\n", (void*)buffer);
```

➢ Not seeing address so :

(gdb) info frame

```
Stack level 0, frame at 0×7fffffffdc80:
 rip = 0×555555555165 in func (stack1.c:10); saved rip = 0×5555555551d2
 called by frame at 0×7fffffffdcb0
 source language c.
 Arglist at 0×7fffffffdc70, args: str=0×55555555601b "hello\n"
 Locals at 0×7fffffffdc70, Previous frame's sp is 0×7fffffffdc80
 Saved registers:
  rbp at 0×7fffffffdc70, rip at 0×7fffffffdc78
```

```
(gdb) info r
rax            0x55555555601b     93824992239643
rbx            0x0                0
rcx            0x555555557dd8     93824992247256
rdx            0x7fffffffddc8     140737488346568
rsi            0x7fffffffddb8     140737488346552
rdi            0x55555555601b     93824992239643
rbp            0x7fffffffdc70     0x7fffffffdc70
rsp            0x7fffffffdc50     0x7fffffffdc50
r8             0x7ffff7f95680     140737353700992
r9             0x7ffff7f96fe0     140737353707488
r10            0x7fffffffd9f0     140737488345584
r11            0x202              514
r12            0x1                1
r13            0x7ffff7ffd000     140737354125312
r14            0x7fffffffddc8     140737488346568
r15            0x555555557dd8     93824992247256
rip            0x555555555165     0x555555555165 <func+12>
eflags         0x206              [ PF IF ]
cs             0x33               51
ss             0x2b               43
ds             0x0                0
es             0x0                0
fs             0x0                0
gs             0x0                0
fs_base        0x7ffff7dab740     140737351694144
gs_base        0x0                0


(gdb) x/2s 0x7fffffffdc80
0x7fffffffdc80: "\270\335\377\377\377\177"
0x7fffffffdc87: ""

(gdb) p &buffer
$1 = (char (*)[12]) 0x7fffffffdc64

(gdb) x/s 0x7fffffffe4b0
0x7fffffffe4b0: "8jBr.agent.DD3WFx1YtO"

(gdb) x/4wx 0x7fffffffe4b0
0x7fffffffe4b0: 0x72426a38      0x6567612e      0x442e746e      0x46573344

(gdb) x/8wx $ebp
0xffffffffffffdc70:   ✗ Cannot access memory at address 0xffffffffffffdc70
```

- 32 bit x86 has “ebp” as base pointer register. On 64 bit x86 64, it’s “rbp”
- On 64-bit linux, the stack frame looks like:
  [rbp] saved RBP
  [rbp+8] saved RIP <- Return address

```
(gdb) x/gx $rbp+8
0x7fffffffdc78: 0x00005555555551d2

(gdb) x/2i 0x00005555555551d2
   0x5555555551d2 <main+56>:    lea    0xe49(%rip),%rax        # 0x555555556022
   0x5555555551d9 <main+63>:    mov    %rax,%rdi
```

```
(gdb) disass func
Dump of assembler code for function func:
   0x0000555555555159 <+0>:     push   %rbp
   0x000055555555515a <+1>:     mov    %rsp,%rbp
   0x000055555555515d <+4>:     sub    $0x20,%rsp
   0x0000555555555161 <+8>:     mov    %rdi,-0x18(%rbp)
=> 0x0000555555555165 <+12>:    lea    -0xc(%rbp),%rax
   0x0000555555555169 <+16>:    lea    0xe94(%rip),%rdx        # 0x555555556004
   0x0000555555555170 <+23>:    mov    %rax,%rsi
   0x0000555555555173 <+26>:    mov    %rdx,%rdi
   0x0000555555555176 <+29>:    mov    $0x0,%eax
   0x000055555555517b <+34>:    call   0x555555555050 <printf@plt>
   0x0000555555555180 <+39>:    mov    -0x18(%rbp),%rdx
   0x0000555555555184 <+43>:    lea    -0xc(%rbp),%rax
   0x0000555555555188 <+47>:    mov    %rdx,%rsi
   0x000055555555518b <+50>:    mov    %rax,%rdi
   0x000055555555518e <+53>:    call   0x555555555030 <strcpy@plt>
   0x0000555555555193 <+58>:    mov    $0x1,%eax
   0x0000555555555198 <+63>:    leave
   0x0000555555555199 <+64>:    ret
End of assembler dump.
```

Address Space Layout Randomization

In order to protect against buffer overflows, most recent operating systems introduce many protection mechanisms, among which the most important one is address space layout randomization (ASLR). Basically, in a system with ASLR, the starting address of the heap and the stack, along with other segments, will be randomized, so it's will be difficult for the attack to know or guess the specific address of any memory segments, say the stack.

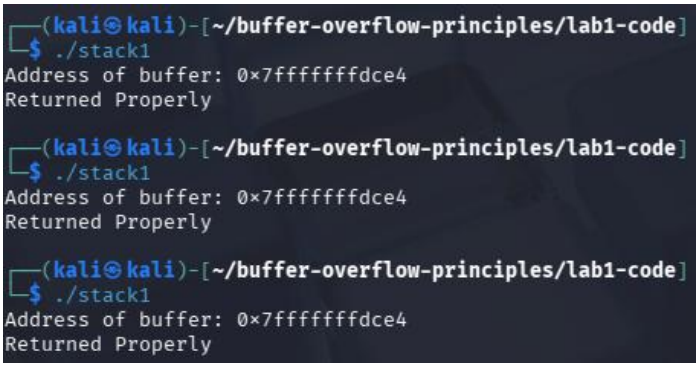
To turn off ASLR, you can run these commands:

```
$ su root
Password : (enter root password)
# sysctl -w kernel.randomize_va_space=0
```

Exercise 3.

Turn off the address space layout randomization, and then do exercise 1 again, write down the three addresses in args.txt, are those three addresses same or not?

Yes they're the same now.



Buffer Overflow

A buffer overflow occurs when data written to a buffer exceeds the length of the buffer, so that corrupting data values in memory addresses adjacent the end of the buffer. This often occurs when copying data into a buffer without sufficient bounds checking. You can refer to Aleph One's famous article to figure out how buffer overflows work.

Now, you run the program stack1, just like below.

```
$ ./stack1 aaaaaaaaaa
Address of buffer: 0x7fffffffdce4
Returned Properly
$ ./stack1 aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
zsh: segmentation fault  ./stack1 aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
```

If you don't observe Segmentation fault, just increase the number of the input a's. Here, the message Segmentation fault indicates that your program crashed due to invalid memory access (for instance, refer to memory address 0).

Exercise 4.

Use gdb, to print the value of the register %eip (64-bit systems use $rip instead) when the program crashes. How does the program run to this address?

```
gdb ./stack1
(gdb) run aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
Starting program: /home/kali/buffer-overflow-principles/lab1-code/stack1
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
Address of buffer: 0x7fffffffdc44

Program received signal SIGSEGV, Segmentation fault.
0x000055555555199 in func (str=0x7fffffffe14b 'a' <repeats 42 times>) at stack1.c:15
15      }

(gdb) info registers rip
rip            0x555555555199      0x555555555199 <func+64>
```

When func() completed, it executed a **ret** instruction, which pops the saved return address of the stack and loads it into the instruction pointer register (**rip**).

Because the buffer overflow already overwrote part of the stack, the value stored in **rip** was corrupted. The CPU's attempt to fetch/execute the instructions at the invalid address (0x555555555199) triggered a segmentation fault & caused the program to crash.

Notes

If gdb is not working, do the following:

```
sudo nano /etc/apt/sources.list
```

In the sources.list file, add the following lines of code at the end.

```
deb http://http.kali.org/kali kali-rolling main contrib non-free non-free-firmware
deb-src http://http.kali.org/kali kali-rolling main contrib non-free non-free-firmware
```

Save the file:

```
> sudo apt update
> sudo apt install gdb
```

Cleanup

- Re-enable ASLR:

  ```
  su root
  sudo sysctl -w kernel.randomize_va_space=2
  ```

  - 0=ASLR off, 1=Partial randomization, 2=Full ASLR (kali's default is 2)